Cryptographic Assurance for Artificial Intelligence: A Framework for Verifiable Content Moderation on Encrypted Channels

1. Introduction

The rapid proliferation of Artificial Intelligence (AI) into critical communications infrastructure has precipitated a fundamental tension between two imperative societal goals: the preservation of digital privacy through strong encryption and the enforcement of safety standards through content moderation. Traditionally, these objectives have been viewed as a zero-sum dichotomy. To moderate content—filtering for hate speech, illicit imagery, or harassment—service providers have historically required plaintext access to user communications, thereby breaking End-to-End Encryption (E2EE). Conversely, maintaining absolute E2EE has often meant relinquishing the ability to govern the interactions occurring within those channels, potentially creating safe havens for malicious activity.

This report proposes a resolution to this dichotomy through a cryptographic framework that leverages Zero-Knowledge Proofs (ZKPs) to establish a regime of "Verifiable AI Governance." Specifically, we analyze the architectural requirements for a system capable of proving that an encrypted message, transmitted over an Advanced Encryption Standard (AES) secured channel, has been processed by a specific content moderation Transformer model and deemed "appropriate," without revealing either the message content or the proprietary parameters of the model.

The core of this proposition rests on the mathematical certification of two distinct computational processes: the decryption of a ciphertext and the inference of a neural network. By combining these proofs using a "Commit-and-Prove" paradigm, we can establish a transitive property of trust. If a user can verify that the input to the AI model is mathematically identical to the plaintext payload of an encrypted message, and that the AI model output a "pass" token, the governance requirement is satisfied without data exposure.

This document serves as a comprehensive research report and architectural blueprint for such a system. It synthesizes the state-of-the-art in Zero-Knowledge Machine Learning (ZKML), reverse-engineers the necessary interoperability standards between cryptographic circuits, and details the specific "binding mechanisms" required to glue standard encryption protocols (AES) to modern deep learning inference engines (Transformers) within a unified proof system.

2. Theoretical Framework: Zero-Knowledge Proofs in Governance

To regulate AI intelligence through cryptography, we must transition from "trusted execution"—relying on the reputation of the entity running the model—to "verifiable execution," where trust is derived from mathematical guarantees.

2.1 The Promise of zk-SNARKs

The foundational primitive for this framework is the Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zk-SNARK). A zk-SNARK allows a Prover (the entity running the AI filter) to convince a Verifier (a smart contract, auditor, or user client) that a statement is true without revealing the "witness" (the private data supporting the statement).

In the context of AI governance, the statement takes the form: "I ran Model M on input x and the result is y." The witness includes the weights of Model M (if proprietary) and the input x (the private message). The "Zero-Knowledge" property ensures confidentiality, while the "Succinctness" property ensures scalability. Verifying a SNARK proof is computationally inexpensive—typically requiring milliseconds and constant data size—regardless of the complexity of the underlying computation. This asymmetry is crucial for blockchain-based governance, where on-chain verification costs must be minimized.

2.2 The Halo2 Proof System

The analysis identifies **Halo2** as the optimal proof system for this architecture. Developed originally by the Electric Coin Company for Zcash, Halo2 has evolved into the *de facto* standard for high-complexity ZK applications, particularly those involving Deep Learning.

Halo2 utilizes **PLONKish arithmetization**, a flexible constraint system that surpasses the limitations of earlier Rank-1 Constraint Systems (R1CS). PLONKish arithmetization distinguishes itself through two features critical for this project:

1. **Custom Gates:** These allow circuit designers to define specialized polynomial constraints for high-degree operations. This is essential for efficiently representing the non-linear activation functions (like GELU or Sigmoid) found in Transformer models, as well as the specific algebraic structure of cryptographic rounds.
2. **Lookup Arguments:** This feature allows a circuit to prove that a value exists within a precomputed table. For AES encryption, which relies heavily on S-Box substitutions (mapping 8-bit inputs to 8-bit outputs), lookup arguments reduce the constraint overhead by orders of magnitude compared to bit-decomposition approaches. Similarly, for AI models, lookups enable efficient quantization and non-linearity approximations.

Halo2 supports different Polynomial Commitment Schemes (PCS). For this framework, the **KZG (Kate-Zaverucha-Goldberg)** commitment scheme is selected due to its constant-time verification and compatibility with the Ethereum Virtual Machine (EVM), enabling on-chain governance registries.

2.3 The Commit-and-Prove Paradigm

A monolithic circuit containing both AES decryption logic and a Transformer model would be prohibitively large, difficult to audit, and memory-intensive to prove. To manage this complexity, we adopt the **Commit-and-Prove** paradigm.

This approach modularizes the system into separate circuits that are cryptographically linked:

- **Circuit A (Encryption Integrity):** Proves knowledge of a key k and plaintext p such that $AES(k, p) = C$ (public ciphertext). Crucially, this circuit also calculates a cryptographic commitment to the plaintext, $Com(p)$, and exposes it as a public output.
- **Circuit B (Intelligence Integrity):** Proves that running the neural network M on input p results in output O. This circuit also calculates $Com(p)$ independently and exposes it.
- **The Glue:** The Verifier accepts both proofs only if the public output $Com(p)$ from Circuit A is bitwise identical to $Com(p)$ from Circuit B.

This architecture decouples the "Encryption" domain from the "Intelligence" domain, allowing each to be optimized independently while maintaining a rigid chain of custody over the data.

3. State-of-the-Art Analysis: Zero-Knowledge Machine Learning (ZKML)

The first leg of our PoC requires proving that a "Filter" model processed the data. This domain, ZKML, has seen explosive growth from 2022 to 2025. We analyze the current landscape to select the appropriate tooling and model architecture.

3.1 Framework Selection: ezKL

The research unequivocally points to **ezKL** as the leading framework for converting general-purpose neural networks into ZK circuits. Unlike other frameworks that might require writing circuits in domain-specific languages (like Circom or Noir), ezKL operates as a transpiler for **ONNX (Open Neural Network Exchange)** files. This allows data scientists to train models in standard environments (PyTorch, TensorFlow), export them to ONNX, and

immediately generate a ZK circuit.

Comparative Advantages:

- **Performance:** Benchmarks indicate ezKL is significantly faster than general-purpose zkVMs (like RISC Zero) for inference tasks. For classical ML tasks like Random Forest or SVM, ezKL has demonstrated proving speeds 28x to 65x faster than RISC Zero. For Deep Learning, ezKL utilizes optimized "gadgets" for matrix multiplication and dot products, which are the computational bottlenecks of Transformers.
- **Quantization Support:** Neural networks operate on floating-point numbers (32-bit or 64-bit floats), while ZK circuits operate over finite fields (prime numbers). ezKL handles the complex process of "quantization"—mapping floats to fixed-point field elements—automatically. It includes calibration tools to determine the optimal scaling factors to minimize accuracy loss, a critical feature for ensuring the ZK "Filter" behaves identically to the Python "Filter."
- **Halo2 Backend:** ezKL generates Halo2 circuits with KZG commitments, aligning perfectly with our selected proof system and ensuring EVM compatibility for the verifier.

3.2 The Classification Model: TinyBERT

While Large Language Models (LLMs) like GPT-4 grab headlines, their parameter counts (billions) make them currently infeasible for ZK proving on consumer hardware due to memory constraints and proving time. For a content moderation use case, "Distilled" BERT models offer the optimal trade-off between accuracy and circuit size.

TinyBERT (specifically the 4-layer or 6-layer variants) is the recommended model architecture for this PoC.

- **Efficiency:** TinyBERT retains over 96% of the performance of BERT-Base on the GLUE benchmark while being approximately 7.5x smaller. This reduction translates linearly to a reduction in circuit constraints, making proof generation feasible in seconds rather than hours.
- **Applicability:** These models are the industry standard for "Discriminative" AI tasks, such as toxicity detection, sentiment analysis, and hate speech classification. They take a sequence of text tokens as input and output a classification probability.
- **Arithmetization:** A TinyBERT model consists primarily of Self-Attention layers and Feed-Forward networks. In ZK, these are represented as massive matrix multiplications. The non-linearities (Softmax in attention, GELU in feed-forward) are the most expensive parts. ezKL handles these via high-fidelity lookup tables, ensuring that the "soft" probability outputs of the Transformer are accurately represented in the proof.

3.3 Handling Inference Privacy

A nuanced requirement of the user's request is proving the filter passed *without* revealing the message. In ezKL, this is managed through **Visibility Settings**.

- **input_visibility**: Set to hashed or private. This ensures the input tokens (the message) are not revealed in the proof.
- **output_visibility**: Set to public. The result of the filter (e.g., 0 for safe, 1 for unsafe) must be public so the smart contract can act on it.
- **param_visibility**: Set to fixed. This commits to the specific weights of the model (e.g., the "Approved TinyBERT Content Filter v1.0") in the verification key. This guarantees that the Prover used the *correct* model and didn't swap it for a dummy model that always outputs "Safe."

4. Verifiable Encryption: Proving AES Correctness

The second leg of the PoC requires proving $h = AES(p, key)$. This is the bridge between the cipherworld (encrypted data on the wire) and the ZK world.

4.1 The AES-ZK Mismatch

Implementing AES in a ZK circuit is notoriously difficult due to a mismatch in "native" operations. AES relies on bitwise operations (XOR, bit-shifts) and byte-level substitutions (S-Box) over the Galois Field $GF(2^8)$. ZK-SNARKs typically operate over a scalar field of a large prime curve (e.g., BN254).

Simulating bitwise XOR in a prime field is expensive. It requires "unpacking" a field element into 254 binary wires, constraining each wire to be 0 or 1, performing the logic, and "repacking" them. A naive implementation of AES-128 can consume vastly more constraints than the neural network itself if not optimized.

4.2 State-of-the-Art Solutions: Lookup Tables

To solve this, modern ZK-AES implementations utilize **Lookup Tables**. Instead of computing the AES S-box or XORs arithmetically, the circuit preloads a table of all possible 8-bit input/output pairs.

- **S-Box Table:** A table with 256 rows mapping input byte x to output byte $S(x)$.
- **XOR Tables:** Some implementations use tables to handle XOR operations on chunks of bits (e.g., 8-bit or 16-bit chunks) to avoid full bit-decomposition.

4.3 Library Selection: Privacy Scaling Explorations (PSE)

The report identifies the **Privacy Scaling Explorations (PSE)** team's Halo2 implementation as the current state-of-the-art. Their halo2-gadgets (and related halo2-wrong repository) provide highly optimized primitives. Specifically, the "AES-128" gadget from the PSE ecosystem or the implementation by **LambdaClass** are the most robust candidates. These libraries provide a "Chip" (a modular circuit component) that takes a key and plaintext as

advice (private inputs) and constrains the instance (public input) to be the valid ciphertext.

4.4 The Binding Requirement

Standard AES gadgets prove $Ciphertext = AES(Key, Plaintext)$. However, this is insufficient. We need to output a commitment to the Plaintext that is compatible with the AI circuit. Standard AES gadgets do *not* output a Poseidon hash of the plaintext; they typically just output the ciphertext.

Therefore, we must **extend** the standard AES gadget. The custom circuit for this PoC must:

1. Ingest Key and Plaintext as private witnesses.
2. Run the AES Gadget to prove Ciphertext.
3. Simultaneously, run a **Poseidon Gadget** on the Plaintext.
4. Expose the Ciphertext AND the PoseidonDigest as public outputs.

This dual-constraint system is the linchpin of the entire architecture.

5. Reverse Engineering the Binding Mechanism

This section addresses the user's specific request to "reverse engineer" the mechanism for proving that the encrypted message corresponds to the filtered message. This is the technical core of the report.

5.1 The "Glue": Poseidon Hash Parameters

The challenge is that the AES circuit and the ezKL circuit are generated by different software stacks. For the commitments to match ($Com_{AES} == Com_{ZKML}$), the hashing algorithms must be identical down to the mathematical constants.

Analysis of ezKL's Hashing Strategy:

Reverse-engineering the ezkl library (specifically looking at src/circuit/modules/poseidon and the poseidon_params.rs files referenced in research) reveals the following configuration:

- **Hash Function:** Poseidon.
- **Field:** BN254 Scalar Field (matching the Halo2-KZG curve).
- **S-Box:** x^α where $\alpha = 5$. This is the standard "friendly" exponent for BN254.
- **Structure:** Sponge Construction.
- **Rate/Width:** This is the critical variable. ezKL typically adapts the width based on input size, but for interoperability, we must fix a standard. Common configurations in the Halo2 ecosystem (used by PSE) utilize a width $T = 3$ (Rate 2) or $T = 5$ (Rate 4). The "Hashed" visibility setting in ezKL defaults to using a Poseidon Sponge that absorbs the

input tensor.

Data Serialization (The "Packing" Problem):

- **AES Side:** The plaintext p exists as a sequence of **bytes** (u8).
- **ZKML Side:** The input p exists as a sequence of **field elements** (Fr), representing token IDs or quantized embeddings.

The Bridge: The AES circuit must perform "Packing." It cannot simply hash the raw bytes if ezKL is hashing field elements. It must group the bytes into field elements *exactly* as ezKL expects them.

- *Scenario:* If the AI model expects Token IDs (integers like 3921, 102), the AES plaintext must be interpreted as a sequence of these integers.
- *Solution:* The AES circuit decrypts to bytes. It then employs a "decomposition gadget" in reverse to pack these bytes into field elements. For example, if token IDs are 16-bit, it packs two bytes into one field element (or one per element).
- *Constraint:* FieldElement_i = Byte_A + 2^8 * Byte_B....
- *Hashing:* These packed Field Elements are then fed into the Poseidon Sponge gadget.

5.2 Implementation of the "Hashed" Visibility

In ezkl, the input_visibility setting controls how the input is handled.

- **Setting:** input_visibility = "hashed".
- **Mechanism:** When this flag is set, ezKL modifies the circuit to:
  1. allocate advice columns for the input (private);
  2. allocate a Poseidon chip;
  3. constrain the input columns to be the preimage of the hash;
  4. expose the **Hash Digest** as the sole public instance column related to input.

To match this in the AES circuit, we must manually construct a Halo2 circuit that mirrors this behavior:

1. **Import:** halo2_gadgets::poseidon.
2. **Configuration:** Initialize the Poseidon chip with the **same** constants (Round Constants, MDS Matrix) as ezKL. These constants are derived from the poseidon_params.rs in the ezKL repo, which usually follows the reference implementation for BN254.
3. **Constraint:** Feed the (packed) plaintext advice columns into the Poseidon chip.
4. **Exposure:** Expose the result as a public instance.

5.3 Public Input Consistency Check

The final step of the binding occurs in the Verifier (Smart Contract).

- **Input 1:** Proof A (AES). Public Inputs: [Ciphertext, Hash_A].

- **Input 2:** Proof B (ZKML). Public Inputs: ``.
- **Logic:**
  ```
  Solidity
  require(VerifierA.verify(proofA, [ciphertext, hashA]));
  require(VerifierB.verify(proofB,));
  require(hashA == hashB, "Commitment Mismatch: Integrity Violation");
  require(result == PASS_VALUE, "Content Filter Failed");
  ```

6. Architectural Convergence: The Commit-and-Prove Paradigm

This section synthesizes the components into a cohesive system architecture.

6.1 The "Filter-and-Forward" Protocol

We define a new protocol for encrypted messaging, the "Filter-and-Forward" protocol.

Actors:

1. **Sender (Alice):** Sends encrypted message C.
2. **Relay/Prover (Bob):** Holds the Governance Key. Can decrypt C, but only inside a ZK circuit. Runs the filter.
3. **Receiver (Charlie):** Wants to receive C, but the network will only deliver it if a proof π is attached.
4. **Verifier (Contract/Network):** verifying the proof.

Note on Privacy: In this specific architecture, the Relay/Prover *does* need the decryption key to generate the proof (since they perform the AES decryption in the witness). This effectively makes the Relay a "Moderator," but one that is **cryptographically constrained**. They cannot modify the message (AES integrity), and they cannot lie about the filter result (ZKML integrity). However, they do see the message in the "witness generation" phase. To achieve *true* side-effect-free privacy where no human *ever* sees the message, the Prover must be a secure enclave (TEE) or the User must generate the proof client-side.

- **Client-Side Proving:** The most privacy-preserving model. The User (Alice) runs the ZKML and AES proofs locally. She sends the Ciphertext + Proof. The network verifies the proof. This guarantees Alice checked her own message against the guidelines before sending. This is the ideal architecture for this PoC.

6.2 Data Flow Diagram

1. **Setup Phase:**

    - **Model Owner:** Trains TinyBERT, compiles with ezKL, deploys Verifier_ML contract and VerificationKey_ML.
    - **Governance:** Deploys Verifier_AES contract.
2. **Proving Phase (Client-Side):**
    - **Input:** Message "Hello World".
    - **Action 1 (AES):** Client generates random key k, encrypts $p \to C$. Generates Proof_AES attesting that C is an encryption of p and $Hash(p) = H$.
    - **Action 2 (ML):** Client runs TinyBERT on p. Generates Proof_ML attesting that $Model(p) = Safe$ and $Hash(p) = H$.
    - **Submission:** Client sends transaction to Smart Contract: {C, Proof_AES, Proof_ML, H, Result}.
3. **Verification Phase (On-Chain):**
    - Smart Contract calls Verifier_AES and Verifier_ML.
    - Asserts H is consistent.
    - Asserts Result == Safe.
    - Emits event MessageApproved(C).

7. Implementation Strategy: The "Filter-and-Forward" Protocol

This section details the practical steps to build the PoC.

7.1 Prerequisite Tooling

- **Rust:** For the AES circuit and Halo2 backend.
- **Python:** For ezKL model compilation and witness generation.
- **Solidity:** For the verification contract.
- **Node.js/SnarkJS:** For frontend integration (optional, but good for testing).

7.2 Detailed Implementation Steps

Step 1: The AI Circuit (ezKL)

1. **Training:** Train a DistilBERT model on the "Toxic Comment Classification Challenge" dataset. Save as .onnx.
2. **Calibration:** Run ezkl calibrate with representative data. This produces a settings.json.
3. **Configuration:** Edit settings.json:
    - Set input_visibility to "hashed".
    - Set output_visibility to "public".
    - Set param_visibility to "fixed".
4. **Compilation:** Run ezkl compile-circuit to generate the .ezkl circuit file and vk (verification

key).

5. **Artifact Extraction:** Crucially, export the **Poseidon Parameters** used by ezKL during this step (often found in the ezKL source or logs) to ensure the AES circuit uses the exact same ones.

Step 2: The AES Circuit (Halo2-Rust)

1. **Scaffold:** Create a new Rust project cargo new aes-verifier.
2. **Dependencies:** Add halo2_proofs, halo2_gadgets (PSE fork), and a local AES implementation (like lambdaclass/AES_zero_knowledge_proof_circuit).
3. **Circuit Definition:**
   - Create a struct AESBindingCircuit.
   - Define Config: AESConfig + PoseidonConfig.
   - **Synthesize Function:**
     - Load private key k and plaintext p into advice columns.
     - Run AESChip::encrypt(k, p) -> get ciphertext_cell.
     - Expose ciphertext_cell to instance column 0.
     - **Packing:** If p is bytes, use a DecompositionChip to pack bytes into Field Elements suitable for the Poseidon hash.
     - Run PoseidonChip::hash(packed_p) -> get hash_cell.
     - Expose hash_cell to instance column 1.
4. **Key Gen:** Run MockProver to verify logic, then generate pk and vk.

Step 3: The Binding Script

1. Write a Python/Rust script to orchestrate the proof generation.
2. Generate random k, p.
3. Compute C and H natively.
4. Run ezKL to generate Proof_ML. Check that the public output matches H.
5. Run Halo2-AES to generate Proof_AES. Check that the public output matches H.
6. If they match, you have a valid proof bundle.

Step 4: The Smart Contract

1. Export Solidity verifiers for both circuits using ezkl create-evm-verifier and the Halo2 keygen_vk tools.
2. Deploy VerifierAES.sol and VerifierML.sol.
3. Deploy Governance.sol which imports both and implements the verify(proofA, proofB, public_inputs) logic.

8. Performance Implications and Scalability

8.1 Constraints and Proving Time

- **TinyBERT in ezKL:** A 4-layer TinyBERT generates a circuit with approximately 20-40 million constraints (depending on sequence length). On a consumer RTX 3090, proving time is in the range of **10-40 seconds**. RAM usage is high, likely 32GB+.
- **AES-128 in Halo2:** A standard AES-128 encryption of one block (16 bytes) requires roughly 20,000 to 30,000 constraints using table lookups. This proves in **< 1 second**.
- **Scalability:** The bottleneck is the Transformer. To scale to longer messages, the AES circuit scales linearly (trivial), but the Transformer circuit scales quadratically (attention mechanism). "Chunking" text into smaller segments and aggregating proofs using recursion is the standard solution for handling long documents.

8.2 On-Chain Gas Costs

Verifying a single Halo2-KZG proof on Ethereum costs roughly 300,000 to 500,000 gas.

- **Naive Approach:** Verify Proof A + Proof B = ~800k gas. This is expensive (~$20-$50 depending on network congestion).
- **Optimized Approach:** Use **Proof Aggregation**. ezKL supports aggregating multiple proofs into a single final proof. The Prover can aggregate the AES proof and the ML proof off-chain, submitting a single succinct proof to the contract. This reduces on-chain cost to ~400k gas total, essentially halving the cost.

9. Security Analysis and Formal Verification

The security of this framework relies on **Soundness** and **Completeness**.

- **Soundness:** A malicious user must not be able to generate a valid proof bundle for a message that fails the filter. This relies on the collision resistance of the Poseidon hash (the "Binding" link). If a user can find p' such that $Model(p') = Safe$ but $Hash(p') = Hash(p_{toxic})$, they can bypass the filter. Poseidon-128 is widely considered secure against this collision attack.
- **Completeness:** A honest user with a safe message must always be able to generate a proof. This relies on the quantization calibration. If the ZK circuit quantizes values differently than the local Python model, valid messages might be rejected.
- **Formal Verification:** The report highlights recent efforts (e.g., by Nethermind) to formally verify Halo2 circuits using the Lean theorem prover. For a high-stakes governance system, formally verifying the "Binding Circuit" (ensuring the packing of bytes to field elements is bijective) would be a critical future step.

10. Conclusion

This report establishes the technical viability of a "Zero-Knowledge AI Governance" framework. By combining the **ezKL** framework for proving TinyBERT inference with a custom **Halo2-AES** circuit for proving encryption integrity, and binding them via a shared **Poseidon commitment**, we can mathematically guarantee that encrypted communications adhere to safety standards without compromising privacy.

While computational overhead currently limits this to smaller "distilled" models and shorter messages, the rapid advancement of ZK hardware acceleration (GPUs/FPGAs) and proof recursion suggests that this architecture will be capable of handling LLM-scale moderation within the next 2-3 years. The Proof of Concept detailed herein serves as the foundational primitive for this new era of "Trustless Moderation."

11. Appendix: Technical Reference Data

Table 1: Comparative Analysis of ZKML Frameworks

Framework	Backend	Primary Use Case	Performance (Inf.)	Quantization	Status
ezKL	Halo2-KZG	Deep Learning (ONNX)	High (Optimized)	Auto-Calibrated	Production-Ready
Modulus	Plonky2	ML (Specialized)	High	Manual	Closed/Service
RISC Zero	STARK	General Purpose (Rust)	Low (Overhead)	None (f32 soft)	Production-Ready
Orion	GKR	ML (Layers)	High	-	Research

Table 2: Poseidon Parameters for ezKL Interoperability (BN254)

Parameter	Value	Note
Field	BN254 Scalar	Matches Ethereum

		Precompiles
Alpha (α)	5	Standard for BN254 S-Box
Full Rounds (R_F)	8	Security against statistical attacks
Partial Rounds (R_F)	56	Security against algebraic attacks
Width (T)	Variable	Typically 3-5 depending on Rate
S-Box	x^5	

Table 3: Circuit Constraints Estimation (PoC)

Component	Constraints (Approx.)	Proving Time (GPU)
AES-128 (1 Block)	~25,000	< 1s
Poseidon Hash	~300	< 0.1s
TinyBERT (4-layer)	~20,000,000	~15-20s
Total System	~20,025,300	~20s

This detailed breakdown provides the roadmap for developers to begin construction of the "Filter-and-Forward" prototype immediately.